

The Sketch Model

A small, fast, on-device vision system that turns what you draw — or photograph — into a running simulation: circuits, mechanics, optics. Built from strokes first, pixels second, and a tiny vision-language model only when nothing else will do.

Written	10 October 2026
For	Rohan Singh (product owner) and any developer joining SIMBLIP
Built by	Claude, developing inside this repository, with Rohan steering
Companion	Document 1 — “The Tutor” (language model, tool layer, teaching engine). Read its Section 5 (runtime and memory) first.
Basis	Read directly from the repo at commit <code>979bc08</code> : <code>lib/sketch/*</code> , <code>app/train</code> , <code>components/workspace/canvas.tsx</code> , <code>lib/scene/circuit-layout.ts</code> , <code>components/objects/part-art.tsx</code> , SimScript and the engines

How to read this document

10-MINUTE READ

Section 1 (what we make), Section 3.1 (the pipeline picture) and Section 4.1 (the five models in one table).

BEFORE WRITING CODE

Section 2 (what already exists — a lot), then the phase you are on in Section 7.

IF YOU ARE ROHAN

Section 5.5 asks you to organise a data-collection session; Section 8 is your learning path; Section 9 lists your decisions.

IF YOU ARE CLAUDE

Section 7 is the work order; Appendix A is the schema every stage must emit.

Contents

- | | |
|--|--|
| 1 What we are making | 6 Training, export, shipping, memory |
| 2 What already exists in SIMBLIP | 7 How to build it: phases and exit criteria |
| 3 Architecture: the pipeline and its contract | 8 What you need to learn, and the tool list |
| 4 The models (S0–S4), one by one | 9 Evaluation, risks, open questions |
| 5 Data: where the training set comes from | A Appendix: schema, taxonomy, worked example, sources |

HONESTY ABOUT NUMBERS

est. marks planning estimates (model-size arithmetic, vendor or press claims) that Phase S2 must replace with measurements. **exists** marks something verified in the repo. Outside facts came from a web search on 10 Oct 2026; sources are in Appendix D. Accuracy targets are *targets*, not results — nothing here has been trained yet.

1. What we are making

1.1 One sentence

You draw it (or photograph it), SIMBLIP understands what the parts are and how they connect, shows its understanding for you to correct, and then it runs — with one undo step to take it all back.

1.2 Three ways in

Input	Example	What the system has to know
A Live ink — strokes from pen, finger or mouse on the canvas	A student sketches a battery, a resistor and an LED and joins them with lines	Which strokes form a symbol; what the symbol is; its orientation; which wire ends touch which pins; any value written beside it ("1k", "9V")
B Photo or screenshot of a diagram	Phone photo of a circuit from a textbook or the whiteboard; a screenshot from a PDF	The same, but from pixels: clean up the picture, find symbols, trace wires, read printed or handwritten labels
C Situation sketches — informal drawings of a physical set-up	A ramp, a block on it, an arrow marked "5 N", "m = 2 kg", " $\mu = 0.2$ "	Objects (ramp, block, spring, pulley), their spatial relations (on, attached to, left of), and the numbers that give them physical meaning

Output for all three: one structured description, the **Scene IR** (Section 3.2), compiled by deterministic code into SimScript — the language the app already runs, lints and renders. The model never writes canvas objects itself.

1.3 Domains, in order of delivery

Wave	Domain	Why this order
1	Analog & digital circuits	Clearest vocabulary (~60 symbols), strict topology (a netlist), the app already auto-draws a clean schematic from a netlist, and the most common thing a student sketches.
2	Mechanics (ramps, masses, springs, ropes, pulleys, pendulums)	Needs spatial reasoning ("touching", "on top of") rather than wiring; the engine already attaches constraints by overlap, so relations are mostly geometry.
3	Optics & waves (sources, lenses, mirrors, slits, screens)	Fewer symbols; left-to-right layout already assumed by the engine.
4	Graphs, trees, state machines, block diagrams (<code>diagram()</code> / DSA Lab)	Different compile target, same recogniser plumbing.

1.4 Why not "just use a vision-language model"?

Question	Giant general VLM	This plan (strokes + tiny specialist models)
RAM	≥ 2 GB for anything usable; competes with the tutor's brain for the same 2 GB	Always-on part is a few MB; optional VLM is loaded alone
Wiring accuracy	Generative models <i>describe</i> ; they hallucinate a plausible but wrong netlist, and you cannot see it	Topology comes from geometry (which end touches which pin) — deterministic and checkable
Verification	None — output is prose	IR $\rightarrow$ compile $\rightarrow$ lint $\rightarrow$ engine run; equivalence can be tested by simulating both
Latency	Seconds per image on CPU	Milliseconds per symbol

Uses the extra information live ink has	No — sees a picture	Yes — stroke order, direction and timing make recognition far easier than pixels
Improves with use	Not without retraining	Every correction is a new labelled example

THE DESIGN STANCE

The recogniser **proposes**, the compiler **decides**, the user **corrects**. This is the same stance `lib/sketch/recognize.ts` already takes: *“a wrong guess costs the user nothing — meaning still comes from behaviors the user attaches.”*

1.5 Success looks like

Dimension	Target (not yet measured)
Symbol accuracy, live ink, writers never seen in training	top-1 $\geq 90\%$, top-3 $\geq 98\%$ on the real (crowd) test set
Whole-circuit correctness (5–12 parts, hand-drawn)	$\geq 80\%$ of circuits compile to a netlist that <i>behaves identically</i> to the intended one (compared by simulating both); the rest need ≤ 2 taps to fix
Photo of a clean printed circuit	$\geq 70\%$ correct on first pass in wave 1; improves with Phase S5
Time	Ink: < 150 ms after pen-up on a Mid device. Photo: < 3 s on Mid, no model download beyond the always-on bundle
Memory	Always-on ≤ 60 MB; photo mode ≤ 2 GB total including the optional VLM, never co-resident with the tutor’s brain
Trust	Never silently wrong: low-confidence parts are highlighted and the user confirms before anything is placed

2. What already exists in SIMBLIP

The sketch-to-simulation idea is **already half built**. This section is the map of what to reuse. All of it was read in the repo.

2.1 The recognition stack today

Piece	File	What it does	Verdict
Shape recogniser exists	<code>lib/sketch/recognize.ts</code> (202 lines)	One stroke → circle / rect / line / regular 3–8-gon / stroke. RDP simplification, “closed” test, deliberately conservative: irregular doodles stay ink.	Keep as S0 . Its restraint is a feature.
Discriminative features exists	<code>lib/sketch/features.ts</code> (335 lines)	<code>normalizeStrokes</code> , <code>stripLeads</code> (removes the shared horizontal leads so the body — zigzag, plates, arrowhead — dominates), <code>features()</code> + a CART decision tree.	The leads insight is gold — carry it into SketchNet’s input design. The tree becomes a fast pre-filter / fallback.
Custom templates exists	<code>lib/sketch/custom.ts</code> (300 lines)	\$P-style point-cloud matching, rotation-sensitive on purpose; templates stored in a global cloud table <code>simblip_sketch_templates (cloud , strokes)</code> so every user’s examples help everyone; two-stage match (tree narrows, body-first distance picks).	This is the baseline to beat and the data source (Section 5).
/train page exists	<code>app/train/page.tsx</code>	Anyone draws a symbol, picks the component it becomes, saves. Multi-stroke. States the goal: “rebuild a whole sketched diagram — every symbol it recognises spawns the real component, and the wires between them already connect via node snapping.”	Upgrade into a guided collection tool (Section 5.5).
Pen + hold exists	<code>components/workspace/canvas.tsx</code> imports <code>recognize</code> and <code>matchCustomSketch</code>	The interaction: draw, hold the pen, shapes and symbols snap to real parts.	SketchNet slots in behind the <i>same</i> gesture. No new interaction to teach.
Part drawings exists	<code>components/objects/part-art.tsx</code>	One SVG body per circuit part, in real pixels, with pins computed by <code>terminalsOf()</code> — “add a part = one entry in ARTS”.	The synthetic-data engine: every class’s clean shape already exists as vector paths.
Auto schematic layout exists	<code>lib/scene/circuit-layout.ts</code>	Simulated-annealing placement (wire length + 400-crossings + perimeter, 20 px clearance) then orthogonal A* routing. Circuits are created <i>without</i> x/y and drawn automatically.	Means the compiler can emit a pure netlist and the app draws it tidily — the user’s messy sketch becomes a clean schematic.

Component registry exists	lib/scene/factory.ts COMPONENTS	The id every class must map to.	Class list is derived from it (no third copy).
SimScript + anchors exists	lib/scene/simscript.ts , simscript-component-reference.md	create(kind, props) , connect(a.anchor, b.anchor) , addproperty, graph.plot; named anchors (.positive .negative .a .b .anode .cathode .base ...).	The IR → SimScript compile target.
Lint / fix / verify exists	simscript-lint.ts, simscript- fix.ts, verify-scene.ts	Static checks and deterministic repairs.	Runs on every compiled sketch — a wrong IR cannot produce a broken scene.
OCR exists	lib/ai/attachments.ts (Tesseract.js, lazy WASM, language data cached in IndexedDB)	Images → text. Reads printed words; poor on diagrams.	Reuse for printed labels in photos (S3b).
Spatial constraint logic exists	docs/architecture.md , physics runtime	"A spring whose endpoint touches a mass attaches to it; a hinge pins whatever bodies it overlaps; nothing found = world anchor."	Mechanics sketches only need <i>correct geometry and classes</i> ; relations come for free.

2.2 What is missing (the real gaps)

1. **No learned recogniser.** Everything today is template or rule matching. It degrades as the number of classes and writers grows — the \$P approach compares to every template and has no notion of "I don't know".
2. **No topology extraction** from messy drawings: wires that overshoot, nearly touch, cross without joining, or have junction dots. Today relies on "node snapping" at placement.
3. **No handwriting or label recognition** — a value written beside a part ("10 kΩ") is ignored.
4. **No photo input.** Images become OCR text (attachments.ts says so plainly: "a circuit diagram yields little or nothing").
5. **No semantics for mechanics sketches** (ramp = static line at an angle, block on it = rigid body, arrow = force), and no way to say "I'm not sure".
6. **No feedback loop.** A user's correction is not captured as a training example.
7. **No evaluation set**, so accuracy can't be stated, only felt.

3. Architecture: the pipeline and its contract

3.1 The pipeline

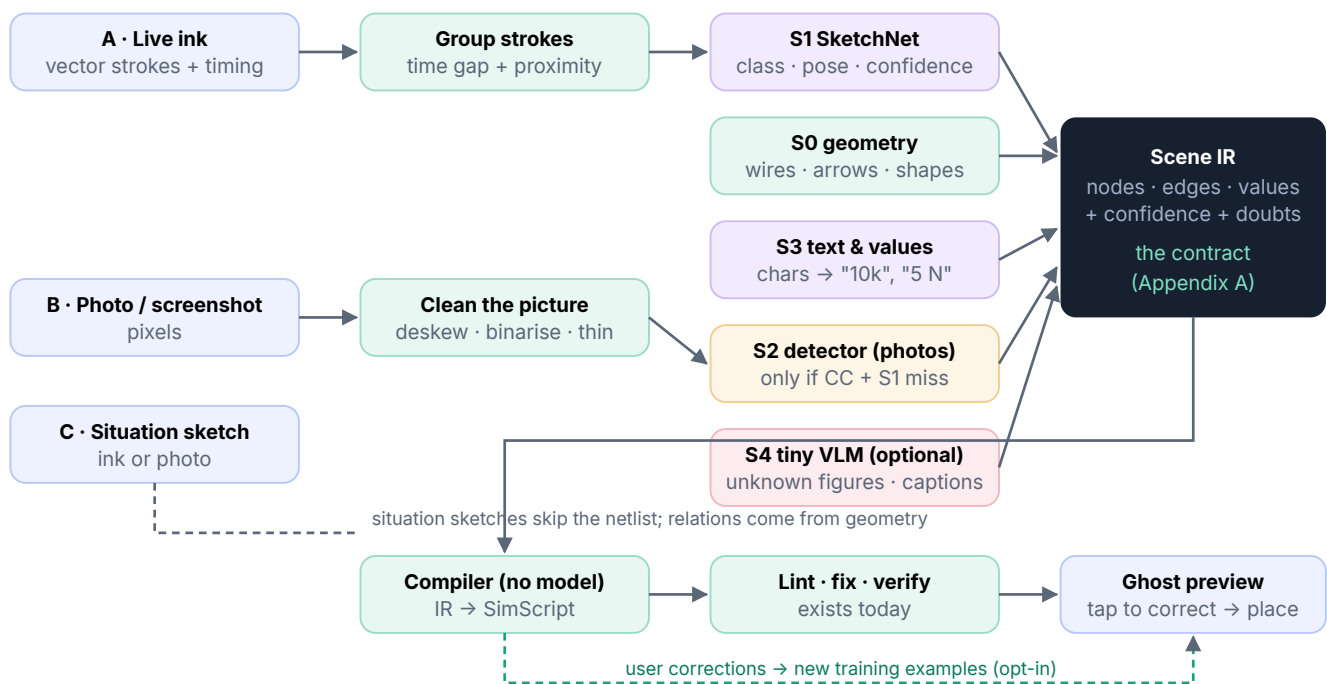


Figure 1 — Every input becomes the same Scene IR; everything after the IR is deterministic and already verified by existing code.

3.2 The contract: Scene IR

The IR is the most important design object in this document. It makes the pipeline testable stage by stage, lets a human or Claude write ground truth by hand, and keeps the models **replaceable**: a better classifier tomorrow changes nothing downstream.

```
// lib/sketch/ir.ts (full zod schema in Appendix A)
type SketchIR = {
  domain: 'circuit' | 'digital' | 'mechanics' | 'optics' | 'diagram'
  nodes: Array<{
    id: string
    cls: string // a COMPONENTS id, e.g. 'resistor', 'battery', 'block'
    conf: number // 0..1, calibrated
    alt?: Array<{ cls: string; conf: number }> // top-3 for the correction menu
    bbox: [x: number, y: number, w: number, h: number]
    rot: 0 | 90 | 180 | 270 // pose; polarity lives here
    flip?: boolean
    values?: Record<string, string> // { R: '10k' } — raw text; the compiler parses units
    source: 'ink' | 'photo'
  }>
  edges: Array<{ a: { node: string; pin: string }; b: { node: string; pin: string }; kind: 'wire' | 'rope' | 'rod' | 'spring' | 'damper'; conf: number }>
  labels: Array<{ text: string; bbox: number[]; attachedTo?: string }>
  doubts: Array<{ ref: string; why: 'low-confidence' | 'ambiguous-junction' | 'dangling-wire' | 'unreadable-value' }>
}
```

Rules: (1) Stages may only *add or refine* IR fields, never invent canvas objects. (2) Every low-confidence decision must also appear in `doubts`, because the UI shows them. (3) The IR is serialisable and small (a few KB) — it can travel to the tutor as part of the `sketch.recognize` tool result.

3.3 The compiler (no model)

Domain	Strategy	Why
Circuit / digital	Emit <code>create()</code> and <code>connect()</code> with no x/y ; the auto-layout engine draws the schematic.	The user's sketch only needs to be <i>topologically</i> right. Hand-drawn geometry doesn't have to be preserved (and shouldn't be — it is untidy). Optional "keep my layout" mode places by IR bbox.
Mechanics	Emit <code>create()</code> with x/y from IR bboxes, attach <code>rigidBody</code> / <code>staticBody</code> by class, set parameters from labels; let the physics runtime resolve constraints by touching.	Position <i>is</i> the information. "Ground below falling bodies" is already a rule in the generator prompt.
Optics	Left-to-right on one axis; y from the sketch; <code>f</code> etc. from labels.	Matches existing engine convention.

The compiler's output goes through `lintSimScript` and `autofixSimScript`. If lint fails, the compiler has a bug, not the model — and the tests (Section 7) catch it on hand-written IR before any model exists.

3.4 Interaction design: "ghost, confirm, place"

1. **Ghost overlay.** After pen-hold (live ink) or import (photo), draw the recognised parts translucent over the user's own ink, each labelled ("resistor 10 k Ω ") with a coloured dot for confidence: green ≥ 0.9 , amber 0.6–0.9, red < 0.6 .
2. **Fix by tapping.** Tap a part $\rightarrow$ the top-3 alternatives plus "draw it again" plus a full list. Tap a wire end $\rightarrow$ "connect / don't connect". Tap a value $\rightarrow$ type it.
3. **Place.** "Make it live" replaces ink with real parts as **one undo step** (the same `pushHistory` guarantee used by AI edits). The original ink is kept on a hidden layer so Undo restores it exactly.
4. **Explain what you see.** The tutor (Doc 1) can narrate the IR: "I read this as a 9 V battery in series with a 1 k Ω resistor and an LED. The LED symbol points the wrong way for current to flow — want to try flipping it?" That is a teaching moment the pipeline gets *for free* because the IR names things.
5. **Learn from the fix.** A correction is stored locally as `{strokes, correctedClass}`; with consent it is uploaded to the same global table `/train` already writes to.

4. The models (S0–S4), one by one

4.1 Summary

Id	Job	Technique	Size	Latency (Mid CPU)	Always loaded?
S0	Stroke grouping, wires, arrows, simple shapes, junction dots	Geometry and rules (existing <code>recognize.ts</code> , extended)	0 MB	< 5 ms	Yes
S1 SketchNet	Symbol class + orientation + confidence from strokes	Tiny CNN on a rendered 32×32 image plus a small GRU on the resampled stroke sequence, two heads	≈ 0.3–1.5 MB int8 est.	< 5 ms per symbol est.	Yes (lazy after first pen-hold)
S3a Ink characters	Handwritten digits, units, Greek, ± etc. beside parts	Same architecture family as S1, ~80 classes, then a unit grammar	≈ 0.5 MB est.	< 5 ms per char	Yes
S3b Printed text	Labels in photos	Existing Tesseract.js	language data cached today	0.5–2 s	On demand
S2 Detector	Find symbols in a messy photo	Nano detector (YOLOX-Nano / NanoDet class), fine-tuned	≈ 1–4 MB est.	≈ 50–200 ms est.	On demand, only if needed
S4 Tiny VLM	Classify an unknown figure, caption it, propose IR for informal pictures	SmoIVLM-256M or 500M	256M reported < 1 GB GPU memory; 500M ≈ 1.23 GB (press figures, unverified)	seconds	No — loaded alone, brain unloaded

4.2 S0 — geometry and rules (extend what exists)

- **Stroke grouping.** Strokes belong to one symbol if they are close in time (pen-up gap < ~600 ms, tuned on data) *and* their bounding boxes overlap or nearly touch. A capacitor is two parallel strokes; a battery several; a gate is a body plus a bubble. Grouping errors are a main failure source, so the grouping parameters are **learned from the collected data**, not guessed.
- **Wires.** Long, near-straight, near-axis-aligned strokes, or polylines with right angles, that are *not* inside a symbol group. Their endpoints are matched to nearby pins (S1 gives a pin template per class; `terminalsOf()` gives real pin positions after placement). Overshoot and gaps below a tolerance snap. T-junctions with a dot connect; crossings without a dot don't (the circuit-redesign phase already added hop arcs for unconnected crossings — the same semantics).
- **Arrows** (forces, velocities, current direction): a line plus a short “V” head at one end; direction and length become an IR edge attribute for mechanics.
- **Ramp, ground, wall:** long strokes classified by angle and by what is attached below/above; hatching (many short parallel strokes) means “fixed ground”.

4.3 S1 — SketchNet

Input design (this is where accuracy is won)

- **Normalise** with the existing `normalizeStrokes`; **strip leads** with the existing `stripLeads` so the discriminating body fills the frame. Feed both the full and the body-only view (two channels).
- **Raster view:** render strokes to 32×32 (anti-aliased, fixed 2 px line width). A tiny CNN handles shape.

- **Sequence view:** resample each stroke to a fixed number of points (e.g. 64 total across strokes) as `(x, y, penUp, direction-change)`. A small GRU captures *how* it was drawn (a zigzag resistor has ≥ 4 direction reversals; a battery has stroke-length asymmetry).
- **Fuse** both embeddings $\rightarrow$ two heads: **class** (~ 100 classes + `unknown`) and **pose** ($0/90/180/270$ + flip). Polarity (diode direction, battery $+/-$) is a pose, not a class — avoids doubling the class count.

Why this should be small

32×32 inputs with ~ 100 classes is MNIST-scale. A 4-block CNN (channels 16-32-64-96) plus a 64-unit GRU is on the order of 150–400 k parameters est. — about 0.3–1.5 MB after int8 quantisation. If it turns out too weak, widen it; there is a hundred-fold margin before it competes with anything else in the memory budget.

Calibration and “I don’t know”

- Train with an explicit `unknown` class (random doodles, letters, partial symbols) and apply **temperature scaling** on a validation set so confidence means something — the UI relies on it for the green/amber/red dots and for the top-3 menu.
- Keep the existing \$P/CART matcher as a **fallback and ensemble member**: if SketchNet’s confidence is low *and* the user has custom templates for this class, use theirs. Users who teach the app their own symbols (the point of `/train`) must never get worse.

Classes

See Appendix B. The list is generated from `COMPONENTS` plus mechanics/optics primitives; **a class without a component to map to is a gap to report, not a class to invent.**

4.4 S3 — text and values

- **S3a (ink).** Segment characters (stroke grouping again), classify with the same architecture over ~ 80 symbols: digits, `. , + - x / ^ = Ω μ π θ k M m n p V A F H Hz s N g °`, a few Latin letters. Then a **unit grammar** turns “1 0 k Ω” into `{R: '10k'}`. Restricting the alphabet is what makes this tiny and accurate; free-form handwriting recognition is a much larger problem and is *not* attempted.
- **Association.** A label belongs to the nearest part (distance in the part’s local frame, preferring “above/below the body”); ambiguous cases go to `doubts` as `unreadable-value` and the UI offers a keypad.
- **S3b (photo).** Tesseract on label crops (not the whole image) with a digits/units whitelist — far more reliable than whole-image OCR, and cheap because crops are small.

4.5 S2 — detector for photos (only if needed)

Strategy: **start without a detector.** A clean textbook diagram can be binarised, split into connected components, and each component classified by S1 (rendered from the component’s pixels). That reuses S1 at zero extra megabytes. Add a nano detector *only* if the Phase S5 evaluation shows recall problems on messy photos (touching symbols, text overlap, shadows).

LICENCE TRAP

The popular Ultralytics YOLOv8/YOLO11 family is released under AGPL-3.0 (with a paid enterprise licence). Shipping it inside SIMBLIP would pull in obligations Rohan almost certainly doesn’t want. Prefer an Apache-2.0 detector (YOLOX-Nano, NanoDet-Plus) and re-check the licence of any pretrained weights before use.

4.6 S4 — tiny vision-language model (optional, last)

SmolVLM-256M and 500M are Apache-2.0 vision-language models published with ONNX and WebGPU demos; a Hugging Face write-up reports the 256M model runs single-image inference in under 1 GB of GPU memory and the 500M model needs about 1.23 GB; the paper reports up to 80 decode tokens/s for the 256M

model in a browser on an M4 Max MacBook. These are **vendor/press numbers**, not CPU measurements on a 4 GB laptop.

Allowed jobs: (a) decide *what kind* of figure this is (circuit? free-body diagram? graph? text-only?) so the right pipeline runs; (b) caption it for the tutor ("a ramp with a block and a pulley"); (c) propose IR for informal situation pictures, **flagged low-confidence**. **Forbidden**: being the only source of wiring.

Memory rule: S4 and the tutor's language model are never resident together. Photo import asks the engine lease to unload the brain, runs S4, releases it, and the brain reloads on the next chat turn (weights stay cached in OPFS). Decide whether S4 ships at all from the Phase S6 evaluation.

5. Data: where the training set comes from

A classifier is only as good as its data, and drawings are where public data is thinnest. The plan uses four sources and — crucially — **never evaluates on synthetic data alone**.

5.1 Source 1 — Synthetic strokes from SIMBLIP's own part drawings

Every circuit part has a vector drawing in `part-art.tsx`; mechanics and optics primitives have shapes in the object renderers. Turn those paths into pen strokes and corrupt them the way real hands do:

```
// scripts/ai/sketch/synth.ts (sketch of the generator)
for each class c, for each style variant v (IEC box resistor, ANSI zigzag, US/IEEE gates, ...):
  paths = svgPathsOf(c, v) // from part-art / object renderers
  repeat N times:
    strokes = splitIntoPenStrokes(paths, strokeCount ~ real distribution, orderPerm)
    strokes = resample(strokes, speedProfile()) // variable point density
    strokes = lowFreqWobble(strokes, amp ~ U(0.5,3)px) // smooth hand tremor (Perlin-like)
    strokes = jitter(strokes, sigma ~ 0.4px) ; overshoot(strokes) ; gapAtJoins(strokes)
    strokes = affine(strokes, rot ~ ±12°, scale ~ U(0.7,1.4), shear ~ ±0.15) // keep 0/90/180/270 poses
explicit
  strokes = maybeDoubleStroke(strokes) ; maybeBreak(strokes)
  leads = addRandomLeads(strokes) // so stripLeads is exercised
  emit {strokes, cls: c, pose}
```

This gives hundreds of thousands of labelled samples for free, **and** it exposes a modelling question worth answering early: *which symbol standard do your students draw?* Resistors appear as zigzags and as boxes across syllabi; both must be classes-in-one (variant augmentation), or the model will say “unknown” to half the room.

5.2 Source 2 — Crowd data from `/train` exists

The `simblip_sketch_templates` table already receives `cloud` and `strokes` from anyone using `/train`. This is the **real-world distribution**: real hands, real devices (mouse, pen, finger). Use it as the fine-tuning set and, above all, as the **test set**, split **by writer** (all examples from one person stay together) so the score means “a person the model never met”.

- Counts matter more than cleverness. Target ≥ 30 writers $\times$ all classes $\times$ 3 repetitions for wave 1 ($\approx 30 \times 60 \times 3 = 5,400$ symbols). That is a classroom afternoon (Section 5.5).
- **Privacy**: strokes only, no identity, no free text. Add a consent line on `/train` saying the examples train a shared model; allow deletion. Students may be minors — get the school's sign-off for any organised session.
- **Quality control**: drop examples that S1 + the old matcher both strongly disagree with the label on (probably mislabelled), then *review* that disagreement list by hand — it is also a list of the confusable classes.

5.3 Source 3 — Public datasets (verify each licence before use)

Use	Candidate	Note
unknown class and generic shapes	Google QuickDraw (stroke-vector doodles, CC BY 4.0 per its release)	Gives millions of non-symbol doodles; attribute properly.
Hand-drawn circuit symbols / diagrams	Published hand-drawn circuit datasets (e.g. CGHD)	Check licence and label format; useful mostly for the photo/detector path.
Handwritten maths characters	CROHME-family online handwriting data	Licence terms vary; digits/units only are needed.

I have not verified these licences. Treat this table as a to-do: **do not train on anything whose licence has not been written into** `docs/ai/decisions.md`.

5.4 Source 4 — Rendered scenes with free labels (for photos)

The app can already generate many valid scenes (the SimScript corpus, the 100-question set, course figures). Because each scene is created *from* a known IR, rendering it to an image gives a picture with **perfect labels for free** (boxes, classes, connections). Then make it look like a photo: paper texture, ruled/grid lines, perspective warp, uneven lighting, shadows, blur, JPEG noise, pen-thickness changes, partial occlusion. Use this to train the detector and to test the clean-up stage. Real photographs are still required for evaluation.

Claude as annotator: Claude can read images, so for a held-out set of ~300 *real* photos (textbook pages, whiteboards, student notebooks — collected with permission) Claude drafts the IR by eye and Rohan or a teacher spot-checks a sample (say one in five). It is slow to label thousands that way but ideal for a small, high-quality test set.

5.5 A data-collection session (Rohan's task)

1. Upgrade `/train` to a **guided mode** (Phase S1): it shows "Draw: resistor" with a progress bar, three repetitions per class, random class order, a device tag (mouse / pen / finger), and a 5-second thank-you screen. Gamify lightly (a leaderboard of classes completed, not of people).
2. Run it in one class period with 20–30 students on their own laptops and phones, with the teacher's agreement and a short consent line.
3. Collect 10–15 **whole hand-drawn circuits and mechanics scenes** per student from a printed prompt sheet ("draw a series circuit with a battery, a switch and two bulbs"). These become the **end-to-end test set** that no model ever trains on.
4. Freeze it: copy to `scripts/ai/sketch/testset/`, hash it, never use it for training.

5.6 Dataset hygiene

- Split **by writer**, never by sample. Split synthetic and real separately and report both.
- Report **per-class** numbers; a single average hides that "capacitor vs battery" is failing.
- Keep a `MANIFEST.json` with source, licence, count and hash for every dataset.
- Version the class list; when a class is added, old models are retrained, not patched.

6. Training, export, shipping, memory

6.1 Where it trains

Everything here fits the author’s **RTX 4060 laptop (8 GB)** — in fact a CPU would nearly do for S1. Tools: **PyTorch** for training, **ONNX** for the exchange format, **onnxruntime** for int8 quantisation and verification, **ONNX Runtime Web** in the browser.

6.2 SketchNet skeleton

```
# scripts/ai/sketch/train/sketchnet.py (skeleton – Claude will complete and test it)
class SketchNet(nn.Module):
    def __init__(self, n_cls, n_pose=8, seq_len=64):
        super().__init__()
        self.cnn = nn.Sequential(
            blk(2, 16), blk(16, 32), blk(32, 64), blk(64, 96), nn.AdaptiveAvgPool2d(1), nn.Flatten())
        self.gru = nn.GRU(input_size=4, hidden_size=64, batch_first=True) # (x, y, pen_up, turn)
        self.cls = nn.Sequential(nn.Linear(96 + 64, 128), nn.ReLU(), nn.Dropout(0.2), nn.Linear(128,
n_cls))
        self.pose = nn.Linear(96 + 64, n_pose)
    def forward(self, img, seq):
        a = self.cnn(img); _, h = self.gru(seq); z = torch.cat([a, h[-1]], 1)
        return self.cls(z), self.pose(z)
# loss = CE(cls, label_smoothing=0.05) + 0.5*CE(pose) ; AdamW, cosine schedule, 30–60 epochs
# augment on the fly: re-run the synthetic wobble on stored vector strokes (never augment a rendered image
only)
# after training: temperature-scale on validation → export ONNX (opset 17) → onnxruntime int8 → check
|logit diff| < ε
```

6.3 Evaluation (what “good” means)

Level	Metric	Notes
Symbol	top-1, top-3, per-class recall, confusion matrix, expected calibration error, unknown precision/recall	Real writer-independent test set reported <i>separately</i> from synthetic
Wiring	edge precision/recall on the whole-scene test set	Counts a wire as correct only if both pins are right
Scene	Behavioural equivalence: compile the predicted IR and the ground-truth IR, run both through the real circuit engine, compare node voltages (circuits) or trajectories (mechanics) within tolerance	Unique to SIMBLIP — the engine <i>is</i> the scorer. Two different-looking netlists that behave identically both pass.
Human effort	taps-to-correct per scene	The metric the user actually feels
Speed & RAM	ms per symbol, MB resident, first-load size	On the three reference devices from Doc 1

Likely confusions to inspect first: resistor ↔ inductor ↔ zigzag-doodle; capacitor ↔ battery ↔ two parallel lines; diode ↔ LED ↔ zener; AND ↔ NAND ↔ bubble-less buffer; mass ↔ wheel ↔ charge; spring ↔ coil inductor; ground ↔ wall hatching.

6.4 Shipping

- **Format:** ONNX int8 files (`sketchnet-v1.onnx` , `inkchars-v1.onnx`), content-hashed, versioned in a small `models.json` manifest shared with the Tutor’s manifest (Doc 1, Section 5.4).
- **Runtime:** ONNX Runtime Web (WASM, SIMD, threads when cross-origin isolated; WebGPU provider when present). Its WASM binary is the real first-load cost (on the order of ten megabytes **est.**) — **lazy-load it**

after the first pen-hold, cache it, and show a small “learning to read your drawing...” first-time state. If that proves too heavy for the always-on feel, S1 and S3a (a few hundred thousand parameters) can be run by a ~150-line hand-written TypeScript forward pass instead; decide with the Phase S2 numbers.

- **Threading:** run in a Worker beside the Tutor’s worker; never on the main thread (physics owns it).
- **Hosting:** the files are small enough to serve from the app’s own static assets; the optional VLM goes to the same object storage as the Tutor’s weights.
- **Credits:** every dataset and model goes into `scripts/third-party-licenses.mjs`, and the “Rohan Singh” credit rule for public surfaces applies to any “powered by” text.

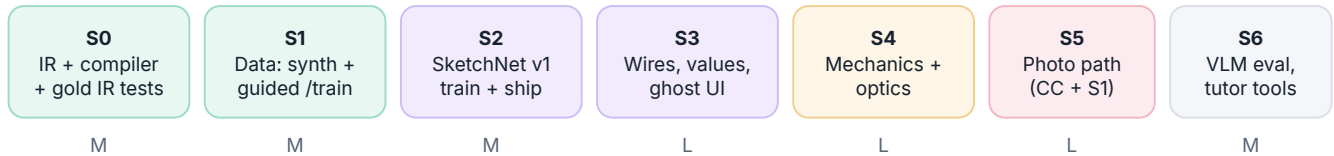
6.5 Memory arbitration with the Tutor

Mode	Resident	Total (est.)
Normal use, Standard tier	Tutor 1.7 B + SketchNet + inkchars + ORT	≈ 1.6–1.7 GB
Normal use, Lite tier	Tutor 0.6 B + SketchNet + inkchars + ORT	≈ 0.8–0.9 GB
Photo import, with VLM	VLM + SketchNet + ORT (Tutor unloaded)	≈ 0.7–1.4 GB depending on 256M / 500M
Photo import, without VLM	Tutor stays; connected components + S1 + Tesseract (transient ≈ 100–200 MB)	≈ 1.9 GB — tight on Standard; use the Lite brain or unload

The Doc 1 `ModelLease` is the one place that decides this; sketch code asks it for permission instead of loading things itself.

7. How to build it: phases and exit criteria

Rule of this plan: **build the pipeline around the models before the models**. A deterministic compiler with hand-written IR can be fully tested on day one; the existing recogniser stands in for S1 until the new one beats it on a fair test.



S0–S1 can start immediately and run beside Document 1's P1–P3. S2 onward needs the shared engine (Doc 1, P4) for memory arbitration.

Figure 2 — Phase order. Sizes S/M/L as in Document 1.

Phase S0 — IR, compiler and gold tests M

- **Build:** `lib/sketch/ir.ts` (zod schema, Appendix A), `lib/sketch/compile.ts` (circuit + digital first), pin-name tables derived from `terminalsOf()` and the anchor list in the SimScript reference, `lib/sketch/compile.test.mjs`.
- **Gold set:** write ~40 IR files by hand (or have Claude write them from textbook circuits and verify) under `scripts/ai/sketch/gold/`: series/parallel, divider, RC, rectifier, transistor switch, op-amp amplifier, gate networks, flip-flop counter.
- **Tests:** every gold IR compiles → `lintSimScript` clean → engine runs; compiled scenes render with the auto-layout engine; wrong IR (missing pin, unknown class) produces a clear error, not a crash.
- **Exit:** 40/40 gold IR compile, lint, and simulate; compile time < 20 ms each.

Phase S1 — Data foundations M

- **Build:** the synthetic stroke generator (Section 5.1) with a CLI that writes a sharded dataset and `MANIFEST.json`; guided `/train` mode; export script for the cloud table → local dataset (writer-split); the class list generated from `COMPONENTS`.
- **Rohan:** run the collection session (Section 5.5); the end-to-end test set is frozen here.
- **Exit:** ≥ 200 k synthetic samples; ≥ 30 writers' real data; the **baseline** (existing CART + \$P matcher) scored on the real test set — this number is what S2 must beat.

Phase S2 — SketchNet v1 M

- **Build:** training script (Section 6.2), calibration, ONNX int8 export, `lib/sketch/sketchnet.ts` (worker wrapper + pre-processing identical to training — a shared test vector file guards against train/serve skew), feature flag in `canvas.tsx` pen+hold path with fallback to the old matcher.
- **Tests:** golden input → golden logits (Python ↔ browser, within tolerance); latency and memory on the reference devices; flag off = behaviour unchanged.
- **Exit:** real-test top-1 beats the baseline and reaches the target in 1.5 (or the gap is explained); < 5 ms per symbol; always-on bundle size recorded and in budget.

Phase S3 — Wires, values, ghost UI L

- **Build:** grouping and wire/junction extraction (S0 extended), S3a ink characters + unit grammar, label association, ghost overlay + tap-to-correct + "Make it live" as one undo step, doubts list, correction capture (local first, opt-in upload).

- **Exit:** on the whole-circuit test set, $\geq 80\%$ behavioural equivalence (or the failure analysis is written and the next fix named); median taps-to-correct ≤ 2 .

Phase S4 — Mechanics and optics L

- **Build:** mechanics classes and relations (ramp, ground hatching, block, ball, spring, rope, pulley if a component exists — otherwise report the gap), arrows $\rightarrow$ forces/velocities, label $\rightarrow$ parameter mapping, compiler paths with x/y; optics left-to-right.
- **Exit:** a set of 30 gold situation sketches ("block on a 30° ramp with friction", "two masses over a pulley", "convex lens with object at $2f$ ") run and match expected motion/ray behaviour.

Phase S5 — Photo path L

- **Build:** clean-up (perspective correction, adaptive threshold, thinning), connected components $\rightarrow$ S1, pixel wire tracing (skeleton + junction detection), S3b label crops, synthetic-photo generator (5.4), real-photo evaluation set labelled with Claude's help.
- **Decision point:** if recall on real photos is poor, fine-tune and ship a nano detector (S2 model in 4.1); otherwise skip it.
- **Exit:** $\geq 70\%$ scene-level correctness on clean printed circuits; transient memory measured; no download beyond the always-on bundle (unless the detector was needed).

Phase S6 — VLM evaluation and tutor tools M

- **Build:** evaluate SmolVLM-256M/500M on 100 real figures for the three allowed jobs (4.6); ship only if it adds measurable value over S5. Implement `sketch.recognize` and `sketch.to_scene` in the Tutor's tool registry (Doc 1, Section 4), with the memory lease. Teaching moves: "here's what I read", polarity and open-circuit mistakes.
- **Exit:** a student can photograph a textbook circuit, see the ghost preview, fix one wire, make it live, and ask the tutor "why does the LED not light?".

7.1 Working agreement for Claude

1. Every stage emits the IR and has a unit test on hand-written input before any model touches it.
2. No metric is reported without the writer-independent real test set. Synthetic numbers are labelled "synthetic".
3. The frozen test sets (`testset/` , hash in `MANIFEST.json`) are read-only in training scripts — enforce with a hash check that fails the run.
4. A feature flag guards every user-visible change; the old recogniser remains the fallback until a phase exit says otherwise.
5. Commit, merge and verify the deployment after each edit (standing rule); append decisions to `docs/ai/decisions.md`.

8. What you need to learn, and the tool list

8.1 Learning path

#	Topic	Depth	Why it matters here
1	What a classifier, a loss, overfitting and a train/validation/test split are	understand	Everything in Section 6.3 depends on it
2	Why split by writer ; leakage; per-class metrics; confusion matrices	understand	The difference between a demo and a product
3	Calibration (“is 90% confident right 90% of the time?”)	understand	The ghost-preview colours are only honest if confidence is calibrated
4	Convolutional networks and GRUs at a conceptual level; parameters vs size on disk; int8 quantisation	understand	To judge the 0.3–1.5 MB claim and to say no to a bigger model
5	Data augmentation; synthetic data and its limits (“sim-to-real gap”)	understand	Why real student drawings are the test set
6	ONNX and ONNX Runtime Web (WASM vs WebGPU provider)	understand / skip internals	How a PyTorch model becomes a browser file
7	Basic image processing: thresholding, connected components, skeletonisation, perspective transform	understand	The photo path; Claude implements, you recognise failures
8	Object detection basics (boxes, IoU, NMS); detector licences	understand	Only if S5 needs a detector
9	Vision-language models: what they can and can’t be trusted with	understand	S4 decision
10	Running a data-collection session ethically (consent, minors, deletion)	use	Section 5.5
11	Circuit/mechanics notation standards (IEC vs ANSI symbols) as your students actually draw them	use	The taxonomy depends on it — you know this better than any model

8.2 Tool and library list

Tool	Purpose	Licence (verify)	Where
PyTorch	Train SketchNet / ink-char models	BSD-style	Train (RTX 4060)
ONNX + onnxruntime (Python)	Export, int8 quantise, verify	MIT	Train
ONNX Runtime Web	Run in browser (WASM / WebGPU)	MIT	Ship
transformers.js	Run SmolVLM in browser (optional)	Apache-2.0	S6, optional
SmolVLM-256M / 500M	Tiny VLM	Apache-2.0 (verify per checkpoint)	S6, optional
YOLOX-Nano / NanoDet-Plus	Nano detector if needed	Apache-2.0 — avoid AGPL YOLOv8/11	S5, conditional
OpenCV.js or hand-written TS	Threshold, components, skeleton, warp	Apache-2.0	S5 (prefer hand-written to avoid a ~8 MB WASM — decide by size)

Tesseract.js	Printed labels	Apache-2.0	Exists
html2canvas-pro	Render scenes to images for synthetic photos	MIT	Already a dependency
Playwright	Latency/RAM harness, synthetic photo capture	Apache-2.0	Already used
QuickDraw / hand-drawn circuit / CROHME-type datasets	Negatives, characters, photos	Check each	Section 5.3

9. Evaluation, risks, open questions

9.1 Risks

Risk	Likelihood	Impact	Mitigation
Real hand-drawing differs from synthetic → good synthetic score, bad real score	High	High	Real writer-independent test set from day one; fine-tune on crowd data; report both; collect more where per-class recall is low.
Look-alike classes confuse students (resistor/inductor, diode/LED)	High	Medium	Pose/feature heads, top-3 menu, tutor asks “is this a diode or an LED?” when the neighbouring value or colour disambiguates.
Wiring errors dominate (wrong junction)	High	High	Deterministic tolerance rules tuned on data; dots mean junctions; ghost preview highlights ambiguous junctions; behavioural-equivalence test shows which errors matter.
Different symbol standards across schools	Medium	Medium	Variant classes folded to one component; survey before fixing the taxonomy.
ORT WASM first-load too heavy for slow networks	Medium	Medium	Lazy load, cache, or hand-written forward pass for S1/S3a.
Photo path under-delivers on messy photos	High	Medium	Honest scope: clean printed diagrams first; guided “retake with the page flat” hints; optional detector/VLM only if measured useful.
Licence problems (AGPL detector, dataset terms)	Medium	High	Licence column in the manifest; no training on unverified data.
Student privacy in uploaded strokes	Low	High	Strokes only, consent, deletion, no identity, school sign-off for sessions.
Memory contention with the Tutor	Medium	Medium	Single lease owner; VLM never co-resident with the brain.

9.2 Open questions

1. **Which symbol standards** do your target colleges actually draw? (A 20-minute survey of two or three teachers answers this and may remove classes.)
2. **Is pen/tablet the main input**, or mouse and touch? Pressure and stroke dynamics differ; the collection session must mirror real use.
3. **Does the VLM earn its place?** Decide from S6 evidence, not enthusiasm.
4. **Which mechanics symbols are missing as components** (pulley, pivot, incline presets)? Each gap is either a new component (which then needs engine support) or an out-of-scope note.

9.3 Decisions that are Rohan’s

- Approving the symbol taxonomy and the standards it covers.
- Organising and consenting the data-collection session.
- Whether student corrections may be uploaded to the shared model by default (recommended: opt-in).
- Whether photo import ships in wave 1 or waits until the live-ink path is excellent (recommended: wait).

Appendix A — Scene IR (full schema)

```
// lib/sketch/ir.ts
import { z } from 'zod'
const Rot = z.union([z.literal(0), z.literal(90), z.literal(180), z.literal(270)])
const Node = z.object({
  id: z.string(),
  cls: z.string(), // validated against COMPONENTS u MECH_PRIMITIVES at compile
time
  conf: z.number().min(0).max(1),
  alt: z.array(z.object({ cls: z.string(), conf: z.number() })).max(3).optional(),
  bbox: z.tuple([z.number(), z.number(), z.number(), z.number()]),
  rot: Rot, flip: z.boolean().optional(),
  values: z.record(z.string(), z.string()).optional(),
  source: z.enum(['ink', 'photo']),
})
const Pin = z.object({ node: z.string(), pin: z.string() }) // pin names = SimScript anchors
(.positive, .a, .anode, .inl ...)
const Edge = z.object({ a: Pin, b: Pin, kind: z.enum(['wire','rope','rod','spring','damper']), conf:
z.number() })
const Label = z.object({ text: z.string(), bbox: z.array(z.number()).length(4), attachedTo:
z.string().optional() })
const Doubt = z.object({ ref: z.string(), why: z.enum(['low-confidence','ambiguous-junction','dangling-
wire','unreadable-value']) })
export const SketchIR = z.object({
  domain: z.enum(['circuit','digital','mechanics','optics','diagram']),
  nodes: z.array(Node), edges: z.array(Edge), labels: z.array(Label), doubts: z.array(Doubt),
})
```

Appendix B — Class taxonomy (wave 1 and 2 starting list)

Ids come from the SimScript `create()` kinds the app already has (see `Modelfile / simscript-component-reference.md`). Anything in *italics* needs a decision (may not exist as a component).

Domain	Classes	Pose-sensitive
Sources and ground	battery, ac-source, current-source, three-phase-source, gnd	battery, current-source
Passives	resistor, bulb, capacitor, inductor, potentiometer, transformer, fuse, switch	capacitor (polarised), switch
Meters	voltmeter, ammeter, wattmeter, probe	—
Semiconductors	diode, led, zener, bjt, bjt-pnp, mosfet, mosfet-pmos, opamp	all
Digital	and-gate, or-gate, xor-gate, nand-gate, nor-gate, not-gate, d-ff, jk-ff, t-ff, sr-latch, mux, demux, half-adder, full-adder, counter4, seven-seg, input, clock, output	all
Mechanics	mass, block, wheel, ground, beam, spring, rope, rod, damper, hinge, motor, charge, <i>incline / wall (static line), pulley, force arrow, velocity arrow</i>	spring/rope are edges; arrows carry direction
Fields	efield, bfield (regions)	direction
Optics	light-source, thin-lens, optical-mirror, optical-screen, slit	lens (converging/diverging), mirror
Waves / quantum	wave-source, wave-boundary, transmission-line, quantum-well, tunnel-barrier	—
Non-symbol	wire (handled by S0), junction dot , text (to S3a), unknown	—

Appendix C — Worked example: from ink to running scene

The user draws: a vertical double-line with the longer line on top (battery), a zigzag labelled “1k”, a triangle-with-bar (diode) pointing the wrong way, and wires joining them in a loop. They hold the pen.

```
// S0 groups 3 symbol-strokes-sets + 4 wires. S1 → [battery 0.97 rot 90, resistor 0.95, diode 0.88 rot 180]
// S3a reads "1 k" next to the zigzag → values {R:'1k'}. Wires end within tolerance of pins → edges.
{
  "domain":"circuit",
  "nodes":[ {"id":"n1","cls":"battery","conf":0.97,"bbox":[40,60,30,60],"rot":90,"source":"ink"},
            {"id":"n2","cls":"resistor","conf":0.95,"values":{"R":"1k"},"bbox":
[120,50,60,24],"rot":0,"source":"ink"},
            {"id":"n3","cls":"diode","conf":0.88,"alt":[{"cls":"led","conf":0.09}],"bbox":
[220,60,40,30],"rot":180,"source":"ink"} ],
  "edges":[ {"a":{"node":"n1","pin":"positive"},"b":{"node":"n2","pin":"a"},"kind":"wire","conf":0.96},
            {"a":{"node":"n2","pin":"b"},"b":{"node":"n3","pin":"anode"},"kind":"wire","conf":0.92},
            {"a":{"node":"n3","pin":"cathode"},"b":
{"node":"n1","pin":"negative"},"kind":"wire","conf":0.95} ],
  "labels":[], "doubts":[{"ref":"n3","why":"low-confidence"}]
}
// Compiler (no model, no x/y → auto-layout draws it):
var n1 = create("battery", {V: 9});
var n2 = create("resistor", {R: 1000});
var n3 = create("diode", {});
connect(n1.positive, n2.a);
connect(n2.b, n3.anode);
connect(n3.cathode, n1.negative);
graph.plot(n2.I);
```

Unspecified battery voltage is the one place the compiler uses a **default** (9 V here) and records it as a doubt so the ghost preview asks “battery voltage?”. Here the diode’s pose (rot 180) places the anode on the resistor side, which is correct for forward conduction when the battery’s positive terminal feeds the resistor; had the user drawn it reversed, the IR would show it faithfully, the simulation would show no current, and the tutor (Doc 1) can turn that into the lesson “why is there no current?”. (*Pin/anchor names above follow the SimScript reference; the exact pose-to-anchor mapping is derived from `terminalstof()` in Phase S0 — treat this example as illustrative.*)

Appendix D — Sources for outside facts (10 Oct 2026)

- [SmoIVLM small variants \(Hugging Face\)](#); [SmoIVLM paper](#); [Gigazine coverage](#); [PyImageSearch: SmoIVLM in the browser with Transformers.js](#) — the memory figures are press-reported and unverified here.
- [Local inference in a browser tab \(2026\)](#) — WebGPU availability varies by browser; sources disagree on Safari/Firefox.
- Dataset and detector licence statements (QuickDraw, hand-drawn circuit sets, CROHME, Ultralytics AGPL) are from general knowledge and **must be re-verified** before use; none were fetched in this session.

Everything about SIMBLIP itself — file paths, behaviour, line counts — comes from reading this repository.